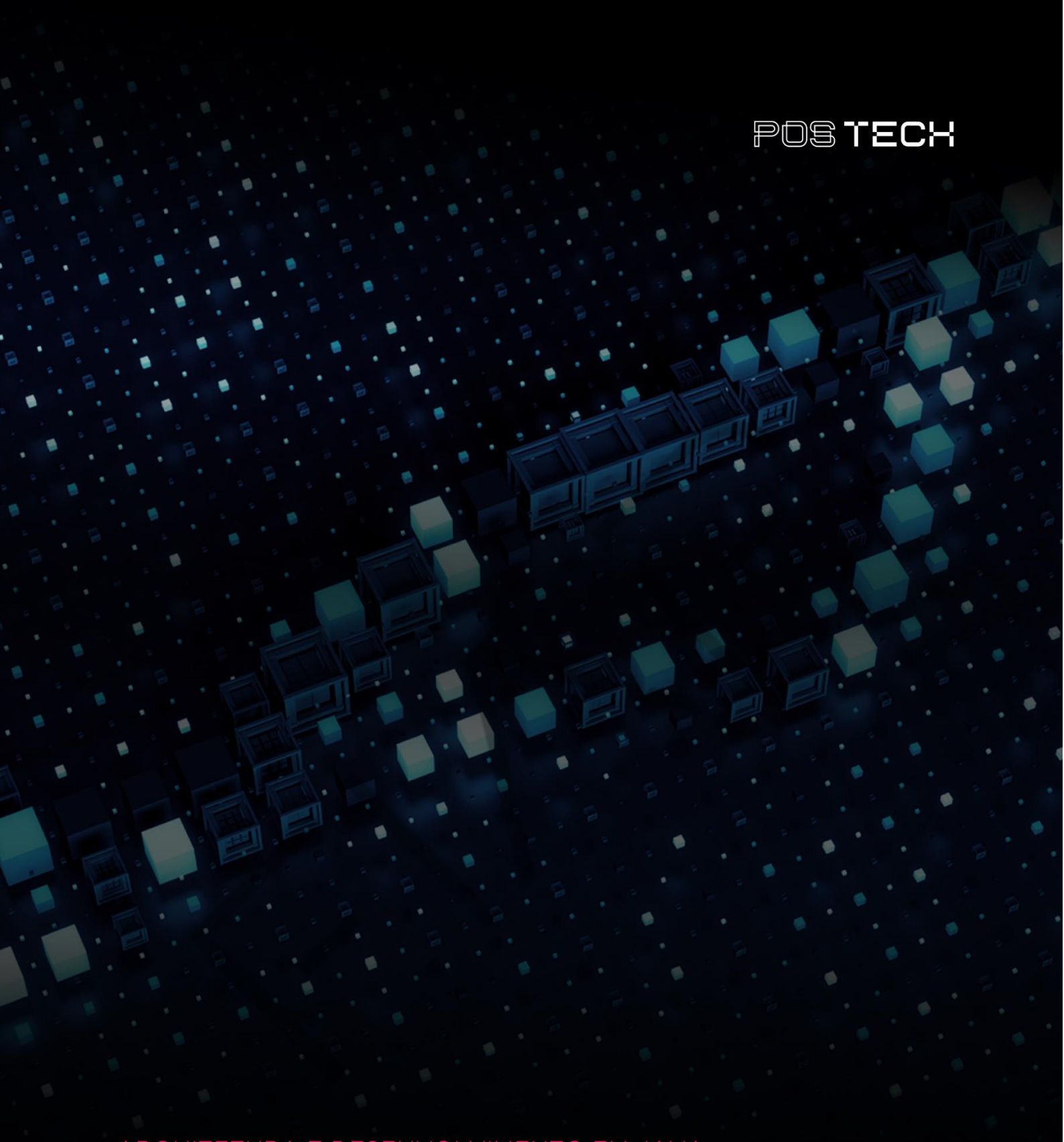


ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 03 -

INTRODUÇÃO À CRIPTOGRAFIA

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 9
REFERÊNCIAS 10
PALAVRAS-CHAVE 11

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Nessa aula, veremos a importância da criptografia na segurança em aplicações Java com Spring Security. Implementaremos a criptografia simétrica e assimétrica em um projeto Spring Boot e utilizaremos o algoritmo BCrypt para hash de senha.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Vamos abordar conceitos importantes para alavancar seu conhecimento sobre a segurança das aplicações e garantir boas práticas em projetos reais. Discutiremos os conceitos de criptografia e demostraremos a implementação de dois tipos principais de criptografia, simétrica e assimétrica, incluindo o algoritmo BCrypt para hash de senha.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

A criptografia é essencial para garantir a segurança da informação, sendo amplamente utilizada para proteger dados em trânsito ou armazenados contra acessos não autorizados. Seu principal objetivo é transformar informações legíveis (**texto claro**) em informações ininteligíveis (**texto cifrado**) e vice-versa, por meio de técnicas matemáticas e computacionais. Essa prática permite garantir a **confidencialidade**, **integridade** e **proteção** dos dados, pilares fundamentais da segurança. Existem dois tipos principais de criptografia: **simétrica** e **assimétrica**.

Na criptografia **simétrica**, o mesmo segredo (uma chave criptográfica) é utilizado tanto para cifrar quanto para decifrar os dados. Por sua simplicidade, ela é conhecida por sua **alta eficiência e velocidade**, sendo ideal para criptografar grandes volumes de dados. No entanto, apresenta desafios para distribuir e gerenciar chaves com segurança, especialmente em sistemas com múltiplos usuários.

Alguns exemplos de algoritmos simétricos são: AES (Padrão de Criptografia Avançada), DES (Padrão de Criptografia de Dados), Blowfish e ChaCha20.

Já a criptografia **assimétrica** utiliza um par de chaves: uma **chave pública**, que pode ser compartilhada, e uma **chave privada**, que deve ser mantida em segredo. A principal vantagem desse modelo é eliminar o problema da troca segura de chaves, pois a chave pública pode ser usada por qualquer pessoa para cifrar uma mensagem, mas **apenas a chave privada correspondente pode decifrá-la**.

Alguns exemplos de algoritmos assimétricos são: RSA (Rivest-Shamir-Adleman), ECC (Criptografia de Curva Elíptica) e Diffie-Hellman.

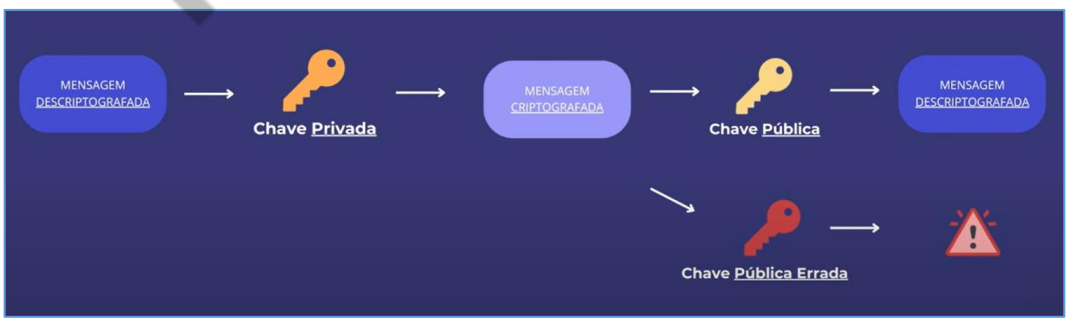


Figura 1 - Criptografia Assimétrica
Fonte: elaborado pelo autor (2025)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Na figura 1, vemos que “**Mensagem descriptografada**” representa uma mensagem legível. Após ser criptografada, os dados serão descriptografados com sua chave correspondente. **“Mensagem criptografada**” mostra a mensagem original transformada em **texto cifrado** após ser **criptografada**. Isso demonstra como a chave é usada para proteger a informação.

Ao tentarmos descriptografar essa mensagem com uma **chave errada** (que não forma par com a chave privada), o processo falha. O resultado é uma mensagem ilegível ou corrompida, destacando a importância do par correto de chaves pública/privada. Com a **chave correta**, temos novamente a **“mensagem descriptografada**”.

Aqui está a implementação da classe utilitária para criptografia simétrica:

```java
package com.fiap.aula3cripto;

import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;

public class SymmetricEncryptionUtil {

    private static final String ALGORITHM = "AES";
    private static final int KEY_SIZE = 128;
    private static SecretKey secretKey;

    static {
        try {
            KeyGenerator keyGenerator = KeyGenerator.getInstance(ALGORITHM);
            keyGenerator.init(KEY_SIZE);
            secretKey = keyGenerator.generateKey();
        } catch (Exception e) {
            throw new RuntimeException("Erro ao inicializar chave AES", e);
        }
    }


    public static String encrypt(String data) throws Exception {
        Cipher cipher = Cipher.getInstance(ALGORITHM);
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);
        byte[] encryptedData = cipher.doFinal(data.getBytes());
        return Base64.getEncoder().encodeToString(encryptedData);
    }

    public static String decrypt(String encryptedData) throws Exception {
        Cipher cipher = Cipher.getInstance(ALGORITHM);
        cipher.init(Cipher.DECRYPT_MODE, secretKey);
        byte[] decodedData = Base64.getDecoder().decode(encryptedData);
        return new String(cipher.doFinal(decodedData));
    }
}
```

Código-Fonte 1 – Criptografia simétrica
Fonte: elaborado pelo autor (2025)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Esse código gera uma **chave secreta** para um algoritmo de criptografia (como **AES**), configurando o tamanho da chave (**KEY_SIZE**) e criando uma chave final (**secretKey**).

Criamos uma instância do objeto **Cipher**, que é a classe responsável por executar operações de criptografia e descritiva. **ALGORITHM** é uma string que define qual algoritmo será usado (por exemplo, "AES"). É necessário para definir como os dados serão transformados, incluindo o algoritmo e o modo de operação (por exemplo, AES no modo CBC).

Inicializamos o objeto Cipher no modo de criptografia (**ENCRYPT_MODE**) e associamos uma chave secreta (**secretKey**) ao algoritmo, que será usada para criptografia. No caso de AES, ela pode ter 128, 192 ou 256 bits.

Convertemos o texto original (**data**) em uma sequência de bytes **data.getBytes()**. Usamos o objeto **Cipher** configurado para criptografar esses bytes, retornando o texto cifrado como um array de bytes (**encryptedData**).

Convertemos o texto cifrado (que é um array de bytes) para uma string codificada em Base64. Como os dados criptografados geralmente contêm caracteres binários não legíveis, **o Base64 modificado os transforma em um formato legível** (uma string) para facilitar o armazenamento ou envio.

Aqui temos a classe utilitária para criptografia assimétrica:

```
package com.fiap.aula3cripto;

import javax.crypto.Cipher;
import java.security.*;
import java.util.Base64;

public class AsymmetricEncryptionUtil {

    private static final String ALGORITHM = "RSA";
    private static KeyPair keyPair;


    static {
        try {
            KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance(ALGORITHM);
            keyPairGenerator.initialize(2048);
            keyPair = keyPairGenerator.generateKeyPair();
        } catch (Exception e) {
            throw new RuntimeException("Erro ao gerar par de chaves RSA", e);
        }
    }

    public static String encrypt(String data) throws Exception {
        Cipher cipher = Cipher.getInstance(ALGORITHM);
        cipher.init(Cipher.ENCRYPT_MODE, keyPair.getPublic());
        byte[] encryptedData = cipher.doFinal(data.getBytes());
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
        return Base64.getEncoder().encodeToString(encryptedData);
    }

    public static String decrypt(String encryptedData) throws Exception {
        Cipher cipher = Cipher.getInstance(ALGORITHM);
        cipher.init(Cipher.DECRYPT_MODE, keyPair.getPrivate());
        byte[] decodedData = Base64.getDecoder().decode(encryptedData);
        return new String(cipher.doFinal(decodedData));
    }

    public static String getPublicKey() {
        return Base64.getEncoder().encodeToString(keyPair.getPublic().getEncoded());
    }
}
```

Código-Fonte 2 – Criptografia assimétrica
Fonte: elaborado pelo autor (2025)

O **bloco de inicialização estática** é responsável por gerar o par de chaves RSA (pública/privada) ao carregar a classe. Utiliza um KeyPairGenerator com algoritmo RSA e tamanho de 2048 bits, garantindo segurança. A inicialização ocorre uma única vez, armazenando as chaves em keyPair. Em caso de falha, lança uma exceção runtime. Ele é essencial para preparar a infraestrutura de criptografia assimétrica.

O **método encrypt(String data)** criptografa dados usando a chave pública RSA. Ele configura um Cipher em modo de criptografia, processa os bytes da string de entrada e retorna o resultado codificado em Base64. É ideal para proteger informações sensíveis antes do envio, lançando exceção se houver falha no processo. A codificação Base64 facilita a transmissão segura em formatos textuais.

O **método decrypt(String encryptedData)** descriptografa dados com a chave privada RSA, decodificando a string Base64 para bytes, inicializando o Cipher em modo de descriptografia e processa os dados cifrados. Ele retorna a informação original em formato legível e é fundamental para recuperar conteúdos criptografados. Exceções indicam problemas como chave inválida ou dados corrompidos.

Por fim, o **método getPublicKey()** retorna a chave pública em formato Base64, permitindo seu compartilhamento seguro. É útil para que terceiros possam criptografar dados destinados ao dono da chave privada. A codificação Base64 garante interoperabilidade entre sistemas. Não lança exceções, assumindo que a chave já foi gerada. Valor essencial para configuração de comunicação assimétrica.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula discutimos a implementação dos algoritmos de criptografia simétrica e assimétrica e comentamos detalhes importantes para codificação dos projetos. Finalizamos a aula com a implementação da classe utilitária para utilizar o algoritmo BCrypt e gerar um hash de senha, útil para salvar em um banco de dados, se for o caso.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

BAELDUNG. **Guia do Baeldung**. Disponível em: https://www.baeldung.com/?s=spring%20security. Acesso em: 08 fev. 2025.

NARDONE, M; SCARIONI, C. **Pro Spring Security: Securing Spring Framework 6 and Boot 3-based Java Applications**. New York: Apress, 2023.

SPRING.IO. **Documentação do Spring Security**. Disponível em: https://spring.io/projects/spring-security. Acesso em: 08 fev. 2025.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Palavras-chave: spring. security. algoritmo. simétrica. assimétrica.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com